

Write a C program to simulate: Producer-Consumer problem using semaphores

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full, mutex;
void produce() {
    int item;
    static int count = 1;

    item = count++;

    sem_wait(&empty);
    sem_wait(&mutex);

    buffer[in] = item;
    printf("Producer has produced: Item %d\n", item);
    in = (in + 1) % BUFFER_SIZE;

    sem_post(&mutex);
    sem_post(&full);
}
void consume() {
    int item;

    if (sem_trywait(&full) != 0) {
        printf("Buffer is empty!\n");
        return;
    }

    sem_wait(&mutex);

    item = buffer[out];
    printf("Consumer has consumed: Item %d\n", item);
    out = (out + 1) % BUFFER_SIZE;
```

```

    sem_post(&mutex);
    sem_post(&empty);
}

int main() {
    int choice;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    while (1) {
        printf("\nEnter 1.Producer 2.Consumer 3.Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                produce();
                break;

            case 2:
                consume();
                break;

            case 3:
                printf("Exiting...\n");
                exit(0);

            default:
                printf("Invalid choice!\n");
        }
    }

    return 0;
}

```

```
"C:\Users\sahne\OneDrive\De  x  +  v
Enter your choice: 1
Producer has produced: Item 2

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 1
Producer has produced: Item 3

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 2
Consumer has consumed: Item 1

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 2
Consumer has consumed: Item 2

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 2
Consumer has consumed: Item 3

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 2
Buffer is empty!

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 3
Exiting...

Process returned 0 (0x0)   execution time : 9.594 s
Press any key to continue.
```

Write a C program to simulate: Dining-Philosopher's problem

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>

#define N 5

pthread_mutex_t forks[N];
pthread_t philosophers[N];

void* philosopher(void* num)
{
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % N;

    while (1)
    {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);
```

```

    if (id % 2 == 0)
    {
        pthread_mutex_lock(&forks[left]);
        printf("Philosopher %d picked up left fork %d.\n", id, left);

        pthread_mutex_lock(&forks[right]);
        printf("Philosopher %d picked up right fork %d.\n", id, right);
    }
    else
    {
        pthread_mutex_lock(&forks[right]);
        printf("Philosopher %d picked up right fork %d.\n", id, right);

        pthread_mutex_lock(&forks[left]);
        printf("Philosopher %d picked up left fork %d.\n", id, left);
    }

    printf("Philosopher %d is eating.\n", id);
    sleep(2);

    pthread_mutex_unlock(&forks[left]);
    pthread_mutex_unlock(&forks[right]);

    printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
}

return NULL;
}

int main()
{
    int i;
    int ids[N];

    for (i = 0; i < N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }

    for (i = 0; i < N; i++)
    {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }
}

```

```

    for (i = 0; i < N; i++)
    {
        pthread_join(philosophers[i], NULL);
    }

    for (i = 0; i < N; i++)
    {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}

```

```

Philosopher 0 is thinking.
Philosopher 2 is thinking.
Philosopher 1 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 0 picked up left fork 0.
Philosopher 3 put down forks 3 and 4.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 4 picked up left fork 4.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 3 is thinking.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 4 put down forks 4 and 0.
Philosopher 4 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 3 put down forks 3 and 4.

```